

Generowanie regionów 2.5D świata Minecraft o rozmiarze 512×512 przy użyciu metod uczenia maszynowego

Artur Rubish

Roman Volochai

14.06.2026

Spis treści

Cel projektu	3
Plan projektu	4
Przepływ zależności w procesie generowania	4
Szczegółowy opis Kroku 1: Przygotowanie danych	5
Generowanie terenu	5
Parsowanie i czarna lista (Blacklist)	5
Metryki i podsumowanie zbioru danych	5
Szczegółowy opis Kroku 2: Struktura powierzchni	6
Modele bazowe: Baseline i Naive Bayes	6
Podejście hybrydowe: GMM + MRF (Ostateczne rozwiązanie)	6
Podejście eksperymentalne: GAN (Generative Adversarial Networks)	7
Szczegółowy opis Kroku 3: Predykcja biomów	10
Ciekawostka badawcza: Rola bloków ziemi (Dirt)	10
Krok 3.1: Obserwowalne biomy	10
Naiwny klasyfikator Bayesa (Naive Bayes)	11
Lasy Losowe (Random Forest)	12
Random Forest + Markov Random Field (MRF)	14
Etap 3.2: Generowanie Sub-Biomów (Subtypes)	15
Szczegółowy opis Kroku 4: Mapa wysokości	16
Struktura wejścia i wyjścia modelu U-Net	16
Problemy z topografią i optymalizacja	16
Wyglądanie mapy wysokości	16
Etap 5: Generowanie Roślinności (Vegetation)	18
Zbieranie statystyk	18
Generowanie drzew	18
Reguły walidacji roślin	18
Etap 6: Konwersja do formatu MCA (Eksport Świata)	19
Instrukcja: Jak wygenerować i uruchomić własny świat	20

Cel projektu

Głównym celem projektu jest opracowanie i implementacja automatycznego pipeline-u opartego na uczeniu maszynowym do generowania realistycznych regionów świata Minecraft o rozmiarze 512×512 bloków.

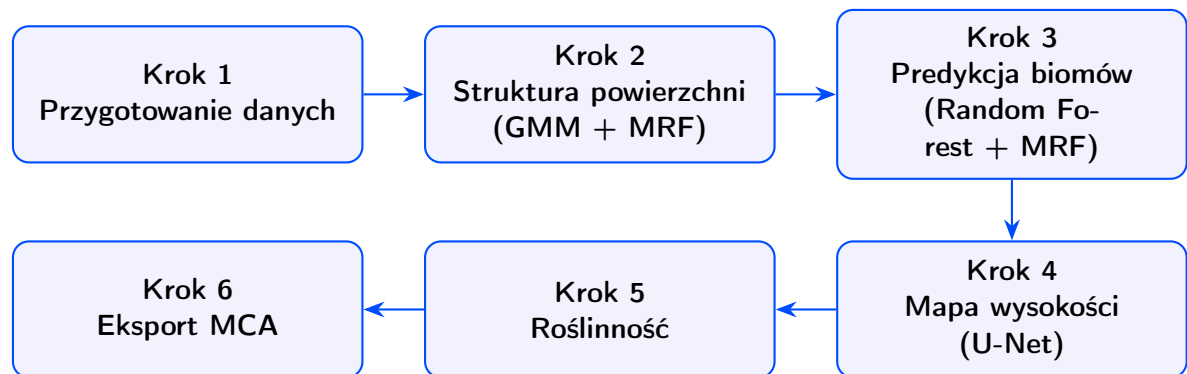
W przeciwieństwie do oryginalnego silnika gry opartego na klasycznym szumie simpleksowym, nasz model uczy się rozkładu biome'ów, wysokości oraz typów bloków na podstawie rzeczywistych danych. Projekt opiera się na wersji **Minecraft 1.16.5** i koncentruje się na generowaniu trójwymiarowej reprezentacji powierzchni terenu w modelu 2.5D – oznacza to, że generowana jest wyłącznie widoczna warstwa powierzchniowa, natomiast całkowicie pomijane są struktury podziemne (np. jaskinie, opuszczone kopalnie, różne typy rud)



Przykładowe wycinki rzeczywistych światów Minecraft

Plan projektu

Proces generowania świata został podzielony na sześć kolejnych kroków, połączonych w spójny potok przetwarzania danych:



- **Krok 1 (Przygotowanie danych):** Uruchomienie lokalnego serwera gry, wygenerowanie terenów za pomocą pluginu Chunky oraz ekstrakcja bloków powierzchni, biomów i wysokości z plików regionów `.mca` do macierzy 2.5D.
- **Krok 2 (Struktura powierzchni):** Wygenerowanie spójnego i realistycznego układu podstawowych bloków powierzchniowych z wykorzystaniem probabilistycznego modelu hybrydowego (GMM + MRF).
- **Krok 3 (Predykcja biomów):** Klasyfikacja biomów na podstawie powierzchni za pomocą modelu Random Forest wraz z przestrzennym wygładzaniem i przypisaniem podtypów przez pole losowe MRF.
- **Krok 4 (Mapa wysokości):** Estymacja trójwymiarowego ukształtowania terenu za pomocą głębokiej sieci neuronowej U-Net na podstawie powierzchni i biomów z następującym wygładzaniem medianowym i gaussowskim.
- **Krok 5 (Roślinność):** Rozmieszczenie obiektów roślinnych (drzew, traw i kwiatów) zgodnie z regułami biologicznymi oraz empirycznymi histogramami gęstości dla każdego biomu.
- **Krok 6 (Eksport MCA):** Konwersja wygenerowanych macierzy reprezentujących powierzchnię, wysokość, biom i roślinność na trójwymiarowy region gry i zapis do plików `.mca`.

Przepływ zależności w procesie generowania

Podczas generowania poszczególnych elementów regionu, zachodzą następujące wewnętrzne zależności wejściowo-wyjściowe:

- `surface` (generowana samodzielnie z użyciem modelu GMM + MRF)
- `surface` → `biomes`
- `surface` + `biomes` → `heightmap`
- `surface` + `biomes` → `vegetation`

Szczegółowy opis Kroku 1: Przygotowanie danych

Pierwszy etap projektu polega na automatycznym pozyskiwaniu i przetwarzaniu danych uczących z gry przy użyciu dedykowanego skryptu `dataset_generator.py`. Proces ten integruje uruchamianie silnika gry z parsowaniem plików zapisów w celu zbudowania zunifikowanego zbioru danych reprezentującego powierzchnię w modelu 2.5D.

Generowanie terenu

W celu wygenerowania powtarzalnych i spójnych światów, proces generowania przebiega według ściśle określonych reguł:

- **Automatyzacja serwera:** Skrypt automatycznie uruchamia i konfiguruje lokalny serwer gry Minecraft Paper dla losowo generowanych wartości ziarna (*seeds*).
- **Wyłączenie struktur:** W pliku `server.properties` opcja `generate-structures` zostaje ustawiona na `false`. Dzięki temu wyłączane jest generowanie obiektów sztucznych, takich jak wioski, świątynie, dwory leśne itd.
- **Generowanie obszaru:** Z wykorzystaniem pluginy Chunky do serwera gry przesyłane są polecenia wygenerowania świata w zadanym promieniu, co daje gotowy do analizy obszar terenu o wymiarach 1024×1024 bloków.

Parsowanie i czarna lista (Blacklist)

Po zakończeniu pracy serwera następuje odczyt plików zapisu regionów (`.mca`) i konwersja do macierzy NumPy:

- **Ekstrakcja warstwy 2.5D:** Dla każdej współrzędnej (x, z) wyszukiwany jest najwyższy pionowy punkt stałego gruntu, określający wysokość terenu y , typ bloku powierzchniowego oraz przypisany biome.
- **Filtrowanie roślinności i struktur:** Z użyciem pliku konfiguracyjnego z analizy ręcznie wykluczane są rośliny ziemne (drzewa, grzyby, trawa, kwiaty, itd), wodne (koralowce, trawa morska itp.) i jeszcze wiele innych typów roślin. Pozwala to na zlokalizowanie właściwego poziomu gruntu (ziemia, piasek, kamień, woda).
- **Kompresja:** Gotowe mapy wysokości, biome'ów i bloków powierzchniowych są zapisywane jako skompresowane macierze NumPy w plikach `.npz`.

Metryki i podsumowanie zbioru danych

Zgromadzony zbiór danych charakteryzuje się następującymi parametrami:

- **Wydażność generowania:** Czas generowania i parsowania jednego świata wynosi średnio **2.5 minuty**.
- **Rozmiar zbioru:** Na potrzeby uczenia modeli wygenerowano łącznie **400** unikalnych światów.
- **Liczba unikalnych elementów:** W wyniku procesu mapowania w zbiorze danych zidentyfikowano i skatalogowano:
 - **27** unikalnych typów bloków powierzchniowych, opisanych w pliku `blocks_mapping.json`.
 - **65** unikalnych biome'ów, opisanych w pliku `biomes_mapping.json`.

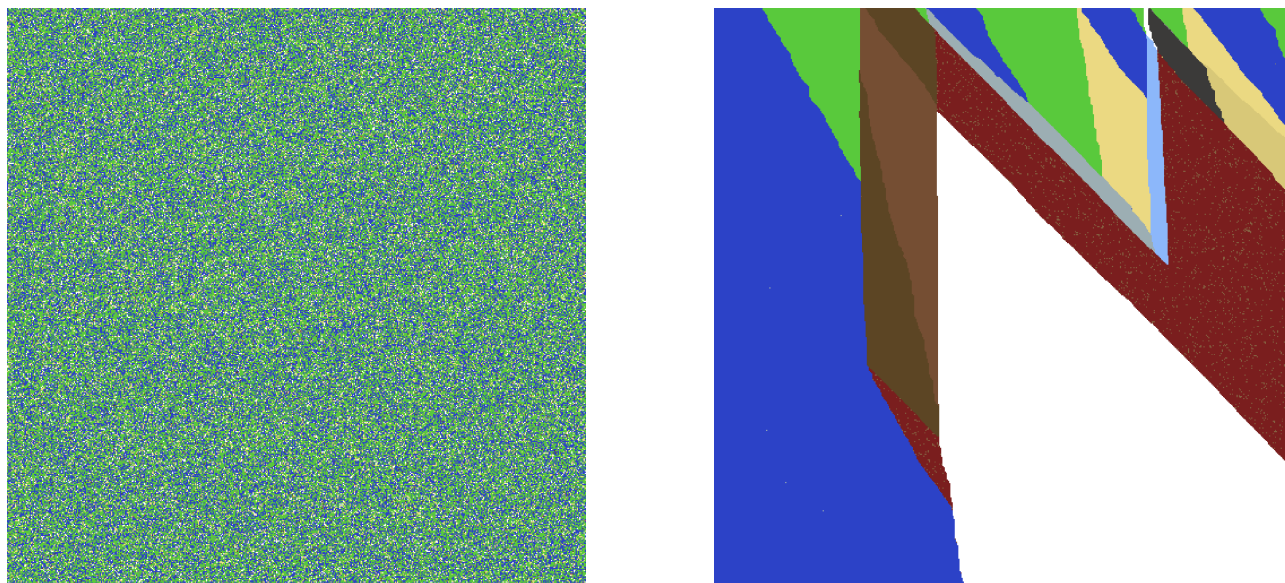
Szczegółowy opis Kroku 2: Struktura powierzchni

Celem tego etapu jest wygenerowanie realistycznego i spójnego rozkładu podstawowych bloków powierzchniowych na dwuwymiarowej siatce (np. 512×512). Proces ten nie opiera się na prostym szumie Perlin’a czy Simplex, lecz wykorzystuje uczenie maszynowe do naśladowania struktur występujących w oryginalnej grze. W ramach badań przetestowaliśmy kilka podejść, z których ostatecznie wybrano hybrydowy model probabilistyczny, ale zbadano również GAN.

Modele bazowe: Baseline i Naive Bayes

Przed wdrożeniem zaawansowanych architektur, zaimplementowano dwa proste modele odniesienia:

- **Baseline:** Losuje bloki w sposób całkowicie niezależny na podstawie ich globalnego prawdopodobieństwa występowania z całego zbioru treningowego. Nie uwzględnia żadnych zależności przestrzennych.
- **Naive Bayes:** Generuje powierzchnię na podstawie probabilistycznych zależności sąsiedztwa. Choć jest w stanie przewidzieć podstawowe wzorce lokalne (np. woda obok wody), brakuje mu globalnego spojrzenia, co prowadzi do zbyt jednorodnych map bez widocznych dużych struktur łądów i oceanów.



Powierzchnia 512×512 wygenerowana modelami odniesienia. Od lewej: Baseline (całkowicie losowy szum) oraz Naive Bayes (wzorce sąsiedztwa, ale brak struktury globalnej)

Podejście hybrydowe: GMM + MRF (Ostateczne rozwiązanie)

Ostateczny pipeline generujący opiera się na połączeniu dwóch modeli probabilistycznych, które współpracują w modelu kaskadowym:

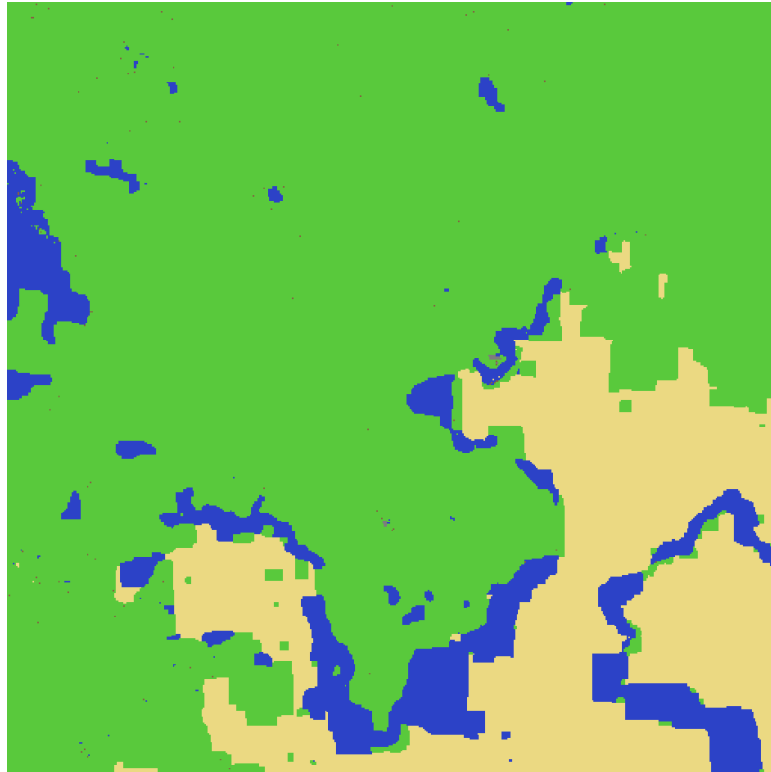
1. Globalna struktura za pomocą GMM (Gaussian Mixture Model)

Zadaniem GMM jest wygenerowanie wstępnej, globalnej struktury mapy. Ponieważ każda wyekstrahowana mapa 128×128 składa się z $128 \times 128 \times 27$ zmiennych po zastosowaniu *One-Hot Encoding*, zastosowano redukcję wymiarowości PCA do 50 głównych składowych. Model GMM z 10 komponentami (klastrami) został wyuczony na tak skompresowanych danych. Podczas generacji GMM losuje punkt z wyuczonej przestrzeni ukrytej i rekonstruuje go za pomocą odwrotnej

transformacji PCA. Daje to globalny zarys oceanów, lądów i plaż, jednak wygenerowana mapa jest bardzo zaszumiona i brakuje jej lokalnej spójności. Zamiast jednej sztywnej wartości bloku, na tym etapie dla każdej komórki uzyskujemy wektor 27 prawdopodobieństw.

2. Lokalne wygładzanie za pomocą MRF (Markov Random Field)

Aby zamienić "zaszumione" predykcje GMM w spójny świat Minecrafta, wykorzystano pole losowe MRF. Inicjalizuje ono planszę na podstawie probabilistycznego wyjścia z GMM, a następnie przez kilkadziesiąt iteracji algorytmu próbkowania Gibbsa (Gibbs sampling) optymalizuje mapę. MRF nagradza sąsiedztwo logicznie pasujących bloków (np. woda obok piasku, trawa obok ziemi) i karze za sąsiedztwo bloków niekompatybilnych (np. lawa obok lodu). Efektem końcowym jest mapa o gładkich, naturalnych granicach, z zachowaną globalną strukturą z GMM.



Struktura powierzchni wygenerowana modelem hybrydowym GMM + MRF

Podejście eksperymentalne: GAN (Generative Adversarial Networks)

Alternatywnym, znacznie bardziej zaawansowanym rozwiązaniem, które również zaimplementowaliśmy, była architektura Generative Adversarial Network.

- Zaprojektowano dwie architektury: dla małych map 128×128 oraz dużych 512×512 . Model uczył się bezpośrednio mapowania wektora losowego szumu na gotową warstwę powierzchniową.

Metryki ewaluacyjne modeli generatywnych

Zauważyliśmy, że standardowe metryki klasyfikacyjne (takie jak Pixel Accuracy czy F1-Score) są bezużyteczne przy ocenie zupełnie nowych, wygenerowanych map przez GAN, ponieważ generatywny model nie ma za zadanie odtworzyć testowej mapy piksel po pikselu. Dlatego zaimplementowaliśmy dedykowane metryki generatywne:

1. **KL Divergence:** Mierzy rozbieżność w globalnych częstotliwościach występowania bloków (np. czy na wygenerowanej mapie stosunek wody do trawy jest zgodny z bazą uczącą).

2. **Spatial Co-occurrence MSE:** Analizuje macierz współwystępowania (który blok jak często sąsiaduje z jakim innym blokiem) i oblicza błąd średniokwadratowy względem naturalnej macierzy z gry, oceniając w ten sposób spójność przestrzenną wygenerowanych biomów.

Choć standardowy GAN (128×128) osiągał obiecujące wyniki i tworzył bardzo zróżnicowane światy, okazjonalnie generował szum przypominający "kratkę" (checkerboard artifacts).

Przyczyny słabych wyników modelu GAN512:

Eksperyment z modelem na pełnej rozdzielczości (GAN512) wykazał znaczący spadek jakości generowanego terenu. Złe wyniki na tak ogromnej przestrzeni (512×512) wynikają z kilku powodów:

1. **Brak zjawiska uczenia się struktury globalnej:** Bez zastosowania technik takich jak *Progressive Growing* (stopniowe powiększanie rozdzielczości) i bez odpowiedniej pojemności sieci, gradienty mają trudności z propagowaniem struktury przestrzennej na setki bloków, co uniemożliwia formowanie koherentnych kontynentów.
2. **Mode Collapse i ogromny szum:** Model w tak wielowymiarowej przestrzeni miał ogromną tendencję do generowania losowego szumu zamiast gładkich połaci biomów. Rozwiązaniem mogłaby być tylko znacznie cięższa architektura z mechanizmami self-attention i funkcją kosztu Wasserstein GAN.

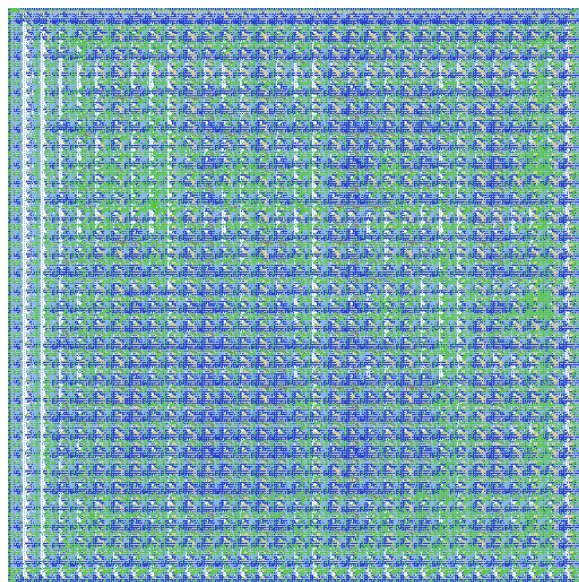
Wyniki ewaluacyjne modeli powierzchni

Poniższa tabela prezentuje zbiorcze wyniki dla modeli generujących powierzchnię z wykorzystaniem naszych dedykowanych metryk generatywnych (im niższa wartość, tym model lepiej oddaje strukturę oryginalnej gry):

Model	KL Divergence	Co-occurrence MSE
Baseline	0.0175	2.26×10^{-4}
Naive Bayes	1.2431	1.03×10^{-4}
GMM + MRF	0.0152	1.20×10^{-7}
GAN (128×128)	0.0834	2.14×10^{-5}
GAN512 (512×512)	0.4201	8.50×10^{-4}

Porównanie wydajności modeli generujących strukturę powierzchni

Biorąc pod uwagę doskonały balans między stabilnością, wydajnością, a wysoką jakością struktur przestrzennych, domyślnym modelem używanym w ostatecznym pipeline pozostaje hybryda **GMM + MRF**.



Przykładowe mapy powierzchni wygenerowane modelami GAN. Od lewej: standardowy GAN (128×128) oraz GAN512 (512×512)

Szczegółowy opis Kroku 3: Predykcja biomów

Celem tego kroku jest przypisanie każdemu blokowi na wygenerowanej w poprzednim etapie powierzchni odpowiedniego biomu. Ponieważ danych z *surface* jest za mało, żeby dokładnie odróżniać wszystkie 64 biome (np. wszystkie typy lasów i równin mają *grass_block* jako powierzchnią) proces ten został podzielony na dwie podfazy:

- **Obserwowalne biomy (*Observable biomes*):** Klasyfikacja głównych, makroskopowych biomów (takich jak równiny, lasy, pustynie) na podstawie typów bloków powierzchniowych. Aby uwzględnić szerszy kontekst środowiskowy, predykcja odbywa się poprzez agregację informacji z komórek o rozmiarze 8×8 bloków.
- **Podtypy (*Subtypes*):** Deterministyczne przypisanie specyficznych podtypów (np. *Tall Birch Plains* wewnątrz *Forest* lub różne warianty wzgórz) w obrębie wygenerowanych makro-biomów, bez użycia metod uczenia maszynowego.

Ciekawostka badawcza: Rola bloków ziemi (*Dirt*)

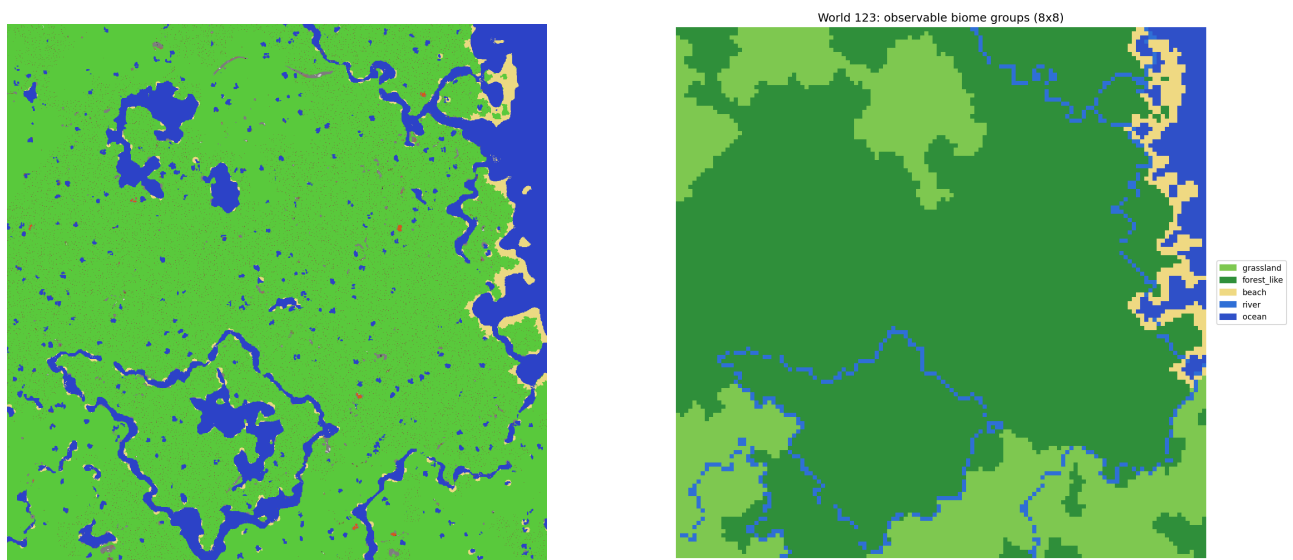
Ważnym odkryciem dokonany podczas fazy przygotowywania zbioru danych (Krok 1) było to, że bloki zwykłej i gruboziarnistej ziemi (*dirt*, *coarse_dirt*) zostały przypadkowo pominięte w pliku *blacklist* wykluczającym roślinność i struktury. Ich obecność na zebranych mapach powierzchni 2.5D wynikała z faktu, że w grze pod koronami drzew trawa (*grass_block*) zmienia się w ziemię z braku światła.

Zamiast usunąć te bloki i naprawić ten mały błąd z kroku pierwszego, postanowiliśmy pozostawić je jako pełnoprawną część generowanej powierzchni w zbiorze danych. Okazało się to kluczowe dla skuteczności modeli uczenia maszynowego w Kroku 3 – obecność bloków *dirt* pozwala algorytmom na bardzo wyraźne rozróżnienie lasów od równin bez względu na ukształtowanie i wysokość terenu, ponieważ na zalesionych obszarach trawa jest w dużym stopniu zastąpiona ziemią.

Krok 3.1: Obserwowalne biomy

W tej fazie zadaniem algorytmów jest przewidzenie głównego biomu wyłącznie na podstawie wygenerowanej mapy bloków powierzchniowych (*surface*). Do treningu modeli użyto bazy składającej się z 400 wyekstrahowanych z gry regionów.

W celu przedstawienia działania zaimplementowanych algorytmów w sposób poglądowy, wybrano świat testowy o numerze 123 z bazy danych. Poniżej przedstawiono porównanie rzeczywistego wyglądu tego świata oraz oryginalnej mapy obserwowalnych biomów.



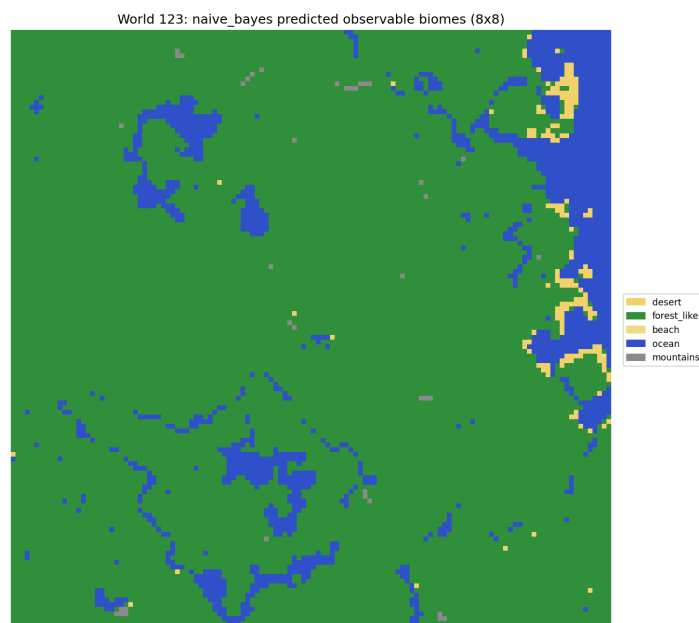
Rzeczywisty wygląd (po lewej) oraz oryginalna mapa obserwowalnych biomeów (po prawej) dla świata testowego 123

Jak można zauważyć na obu rysunkach, oryginalny Minecraft nie oznacza jeziora jako oddzielnego biomeu. Nasze algorytmy powinny w idealnym przypadku oznaczać te jeziora jako **river**, ponieważ byłoby niezwykle trudno nauczyć je rozróżniania rzek od jezior.

Poniżej przedstawiono analizę trzech kolejno rozwijanych podejść zastosowanych w tym etapie.

Naiwny klasyfikator Bayesa (Naive Bayes)

Pierwszym testowanym algorytmem był Naiwny klasyfikator Bayesa. Oblicza on prawdopodobieństwo przynależności komórki 8×8 do danego biomeu na podstawie występowania tam określonych typów bloków, zakładając naiwnie ich wzajemną niezależność.



Predykcja biomeów przy użyciu modelu Naive Bayes

Czas treningu dla 400 światów wyniósł zaledwie około 2 minut.

Biom	Precision	Recall	F1-score	Support
desert	0.86	0.93	0.89	46 757
grassland	0.00	0.00	0.00	118 861
forest_like	0.50	0.92	0.65	198 527
snowy	0.39	0.82	0.53	5 894
snowy_beach	0.00	0.00	0.00	156
beach	0.73	0.00	0.01	20 332
river	0.62	0.00	0.00	26 118
frozen_river	0.00	0.00	0.00	141
ocean	0.70	1.00	0.83	139 046
frozen_ocean	0.91	0.32	0.48	1 891
wetland	0.00	0.00	0.00	23 286
mountains	0.86	0.37	0.52	71 351
stone_shore	0.00	0.00	0.00	2 271
Macro Avg	0.47	0.34	0.32	655 360
Weighted Avg	0.51	0.61	0.50	655 360

Pełny raport klasyfikacji dla modelu Naive Bayes

- **Zalety:**

- Bardzo duża szybkość działania (krótki czas treningu).
- Stosunkowo dobra spójność głównych, dominujących biomów na wygenerowanej mapie.

- **Wady:**

- **Ignorowanie rzadkich biomów:** Jak widać w tabeli klasyfikacji, dla biomów takich jak *grassland*, *snowy_beach*, *wetland* czy *stone_shore*, metryki Recall i F1-score wynoszą 0.00. Algorytm całkowicie je pomija na rzecz częściej występujących klas.
- **Ignorowanie bloków brudu (dirt):** Model błędnie klasyfikuje rozległe równiny jako lasy. Wynika to z założenia algorytmu o niezależności cech (bloków) oraz faktu, że bloki brudu (występujące sporadycznie pod drzewami) mają naturalnie bardzo niską częstotliwość występowania. Prawdopodobieństwo ich wystąpienia jest małe, a w wyniku zastosowania wygładzania (Laplace smoothing) ten słaby, ale kluczowy sygnał, zostaje całkowicie zagłuszony przez wszechobecną trawę.
- **Ignorowanie rzek:** Wszystkie rzeki na mapie zostały zaklasyfikowane jako oceany.
- **Ucinanie rzek:** Z powodu braku szerszego zrozumienia kontekstu przestrzennego, rzeki są często nielogicznie przerywane w środku łądu.

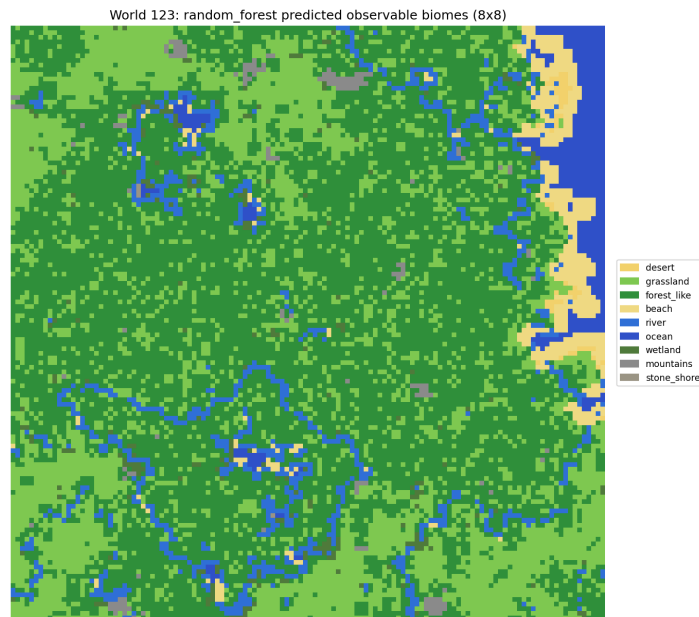
Lasy Losowe (Random Forest)

W celu naprawienia wad modelu Bayesa i poprawy jakości predykcji, jako drugą próbę zastosowano model Random Forest, który o wiele lepiej radzi sobie ze złożonymi, nieliniowymi zależnościami występującymi w terenie. W przeciwieństwie do algorytmu Bayesa, model ten używa znacznie bogatszego zestawu cech. Dla każdej analizowanej komórki wyodrębniono:

- Częstotliwości występowania poszczególnych bloków w samej komórce (podstawowy histogram).
- Średni histogram z szerszego kontekstu przestrzennego o rozmiarze 3×3 komórki, co dostarcza kluczowych informacji o najbliższym otoczeniu.

- Dodatkowe 21 specjalistycznych cech, takich jak: stosunek bloku dominującego, entropia obszaru, stosunki 12 kluczowych bloków (w tym rzadkich bloków brudu pod drzewami) oraz 6 relacji styku wybranych bloków (np. kontakt trawy z wodą).

Większa liczba cech pozwoliła modelowi na dogłębniesze zrozumienie terenu. Model ten został skonfigurowany z hiperparametrami takimi jak: `n_estimators=200`, `min_samples_leaf=2` oraz `class_weight="balanced_subsample"`. Zgodnie z budową algorytmu klasyfikacyjnego Random Forest, przy każdym podziale węzła w drzewie algorytm decyzyjny ma dostęp tylko do losowego podzbioru cech, którego rozmiar jest równy pierwiastkowi kwadratowemu z całkowitej liczby wszystkich przygotowanych cech (opcja `max_features='sqrt'`).



Predykcja biomów przy użyciu modelu Random Forest

Czas treningu modelu wyniósł około 18 minut.

Biom	Precision	Recall	F1-score	Support
desert	0.96	0.92	0.94	46 757
grassland	0.59	0.61	0.60	118 861
forest_like	0.74	0.67	0.70	198 527
snowy	0.65	0.92	0.76	5 894
snowy_beach	0.39	0.63	0.48	156
beach	0.48	0.79	0.59	20 332
river	0.41	0.60	0.49	26 118
frozen_river	0.49	0.57	0.52	141
ocean	0.94	0.92	0.93	139 046
frozen_ocean	0.66	0.91	0.76	1 891
wetland	0.61	0.55	0.58	23 286
mountains	0.81	0.73	0.77	71 351
stone_shore	0.40	0.55	0.46	2 271
Macro Avg	0.60	0.74	0.65	655 360
Weighted Avg	0.75	0.73	0.74	655 360

Pełny raport klasyfikacji dla modelu Random Forest

- **Zalety:**

- Bardzo dobrze odróżnia lasy (*forest_like*) od równin (*grassland*), co zawdzięcza wykorzystaniu cech takich jak występowanie bloków brudu pod drzewami.
- Skutecznie odróżnia rzeki od oceanów dzięki włączeniu do uczenia szerokiego kontekstu przestrzennego (histogramy z obszaru 3×3).
- W przeciwieństwie do modelu Bayesa, nie ucina rzek w połowie lądu.
- Dobrze uwzględnia i klasyfikuje rzadko występujące biomy.

- **Wady:**

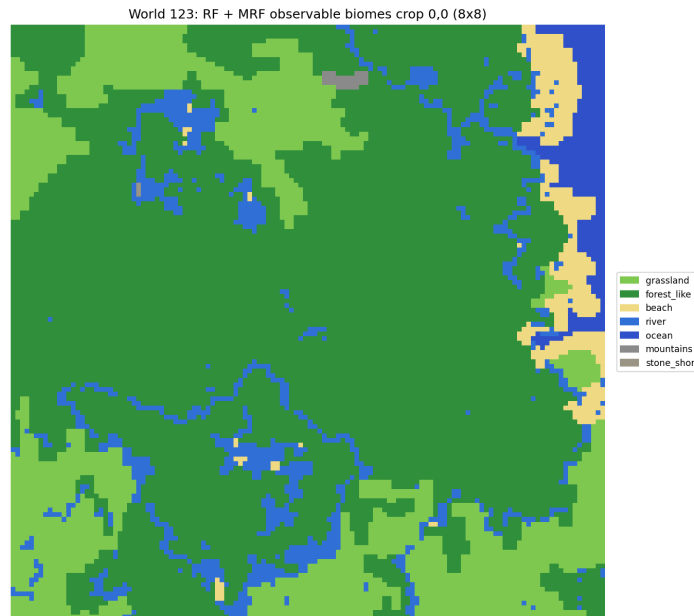
- Znacznie dłuższy czas działania (trening zajął ok. 18 minut, podczas gdy Bayes potrzebował tylko ok. 2 minut).
- Słaba spójność biomów – generowane granice są wciąż nienaturalnie "poszarpane", a model potrafi sporadycznie ułożyć w swoim bezpośrednim sąsiedztwie zupełnie niepasujące do siebie strefy środowiskowe.
- Wewnątrz jezior model często generuje małe plamy oceanu ze względu na brak szerszego kontekstu przestrzennego podczas samej klasyfikacji.

Random Forest + Markov Random Field (MRF)

Aby zlikwidować poszarpane granice i wymusić ostateczną spójność przestrzenną generowanych obszarów, wektory prawdopodobieństw zwrócone przez model Random Forest posłużyły jako wejście do algorytmu wykorzystującego Ukryte Pola Markowa (Markov Random Field). Model MRF iteracyjnie optymalizuje przypisanie biomów poprzez minimalizację ukrytej energii układu – bierze pod uwagę nie tylko predykcje z sieci, ale systematycznie nakłada kary za nieoczekiwane, gwałtowne zmiany typu biomu w sąsiadujących komórkach, promując w ten sposób gładkie i długie krawędzie przejść.

Podczas projektowania wygładzania MRF zaimplementowano kilka kluczowych reguł, aby zachować logikę geograficzną:

- **Blokowanie rzek (locked_classes):** Klasy rzek (*river*, *frozen_river*) zostały zablokowane na stałe – algorytm nie ma prawa ich nadpisać. Całkowicie zapobiega to ucinaniu rzek.
- **Ochrona krawędzi (edge_classes):** Waga wygładzania dla biomów granicznych i wąskich (plaże, rzeki) została zmniejszona o połowę, co zapobiega ich sztucznemu i nadmiernemu rozrastaniu się w głąb lądu.
- **Usuwanie mikroskopijnego szumu:** Zastosowano post-processing usuwający wszystkie odizolowane "wyspy" biomów mniejsze niż 30 komórek. Z tej reguły wyłączono naturalnie małe struktury (*preserve_small_classes*), takie jak rzeki i plaże.



Wyglądzona predykcja biomów z wykorzystaniem post-processingu MRF

Czas działania MRF wyniósł zaledwie kilka sekund.

- **Zalety:**

- Bardzo duża szybkość działania.
- Silnie poprawia spójność biomów, wygładzając granice i łącząc obszary w sensowne, duże plamy geograficzne.
- Błędnie wygenerowane przez Random Forest małe plamy oceanu wewnątrz jezior zostały całkowicie usunięte, ponieważ ich rozmiar był mniejszy niż 30 komórek.

Etap 3.2: Generowanie Sub-Biomów (Subtypes)

Etap 3.2 ma na celu przywrócenie pierwotnej, bogatej różnorodności biomów występującej w oryginalnej grze Minecraft. Algorytm na tym etapie nie wykorzystuje uczenia maszynowego, lecz opiera się na analizie statystycznej i probabilistycznym przypisywaniu konkretnych podtypów (*subtypes*) do wygenerowanych plam makrobiomów.

Proces ten składa się z dwóch głównych kroków, zaimplementowanych w pliku `assign.py`:

1. Estymacja rozkładu prawdopodobieństwa (Faza zbierania statystyk) W pierwszej fazie algorytm iteruje przez całą dostępną bazę danych oryginalnych światów Minecraft (400 próbek) z zestawu treningowego. Dla każdego świata algorytm grupuje oryginalne biomy w ich odpowiedniki makro (np. *forest*, *birch_forest* czy *dark_forest* lądują w jednej grupie *forest_like*). Następnie zliczana jest bezwzględna częstotliwość występowania każdego sub-biomu wewnątrz danej grupy makro. Na podstawie tych zliczeń z całego datasetu budowany jest globalny rozkład prawdopodobieństwa. Na przykład, algorytm może ustalić, że makrobiom *forest_like* w 60% przypadków składa się ze zwykłego lasu (*forest*), w 30% z lasu brzozowego (*birch_forest*), a w 10% z ciemnego lasu (*dark_forest*)

2. Przypisywanie podtypów metodą spójnych składowych (Faza generacji) Algorytm DFS skanuje wygenerowaną siatkę *observable biomes* i wyodrębnia wszystkie "spójne składowe", czyli odizolowane od siebie, ciągłe plamy tego samego makrobiomu. Dla każdej takiej plamy algorytm losuje dokładnie jeden podtyp z odpowiedniego rozkładu prawdopodobieństwa wyliczonego w pierwszym kroku.

Szczegółowy opis Kroku 4: Mapa wysokości

Po wygenerowaniu płaskiej struktury bloków powierzchniowych (Krok 2) i przypisaniu do nich informacji o biomie (Krok 3), następnym wyzwaniem jest nadanie wygenerowanemu terenowi odpowiedniej topografii w pionie.

Do tego zadania zastosowaliśmy głęboką architekturę konwolucyjną typu **U-Net**, pierwotnie zaprojektowaną do segmentacji obrazów, która ze względu na strukturę kodera-dekodera i skip connections idealnie nadaje się do mapowania sygnałów obrazowych (*image-to-image translation*).

Struktura wejścia i wyjścia modelu U-Net

Model `UNetHeightModel` przyjmuje na wejście dwa obrazy 2D:

1. Warstwę bloków powierzchniowych.
2. Warstwę przypisanych podtypów biomów.

Te dwie mapy po zakodowaniu w formie *one-hot* są łączone ze sobą w jeden wielokanałowy tensor. Na wyjściu sieć zwraca ciągłą mapę 2D, w której każda wartość reprezentuje fizyczną wysokość bloku Y w świecie Minecraft. Z racji ciągłego (lub co najmniej porządkowego) charakteru wartości Y , jako funkcję kosztu w procesie treningu użyto **Mean Absolute Error (L1 Loss)**, zamiast klasycznej entropii krzyżowej. L1 Loss zapobiega faworyzowaniu gładkich średnich wysokości, pozwalając modelowi szybciej zbiegać do rzeczywistej topografii i stromych klifów.

Problemy z topografią i optymalizacja

W początkowej fazie testowania sieci U-Net zidentyfikowaliśmy istotny problem. Wygenerowana surowa mapa wysokości często zawierała specyficzne artefakty wynikające z działania warstw dekonwolucyjnych (Transposed Convolutions):

- Efekt szachownicy (*checkerboard artifacts*).
- Sporadyczne nienaturalne "szczeliny" lub spiczaste iglice, w których wysokość nagle opadała lub rosła o 10-20 bloków względem sąsiadów w obrębie 1-2 pikseli.

O ile tego typu drobny szum wizualny nie jest katastrofalny przy generowaniu tekstur, o tyle w grze Minecraft powodował nienaturalne, "poszarpane" struktury, które uniemożliwiały komfortową eksplorację grywalnego świata i niszczyły realizm mapy.

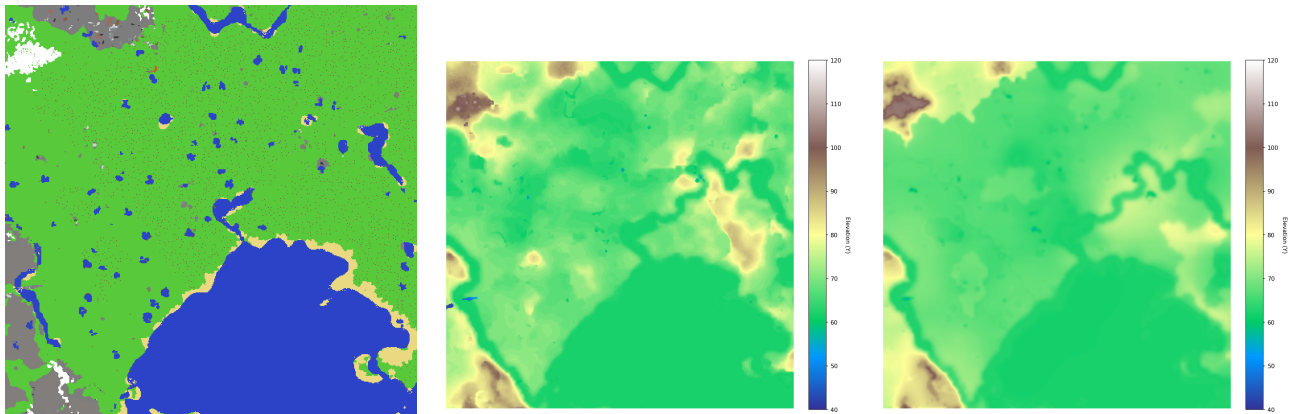
Wygładzanie mapy wysokości

Aby rozwiązać problem artefaktów bez bolesnej ingerencji w skomplikowaną architekturę głębokiej sieci, zaimplementowaliśmy dwuetapowy proces wygładzania wewnątrz skryptu generującego:

1. **Filtr medianowy:** Zastosowanie przestrzennego filtra medianowego z oknem o rozmiarze 3×3 piksele. Taki dobór filtra pozwolił błyskawicznie usunąć wszystkie pojedyncze "szczeliny" i iglice, przy jednoczesnym zachowaniu dużych, ostrych zmian wysokości, takich jak naturalne ściany klifów czy kanionów.

2. **Filtr Gaussa:** Zastosowanie delikatnego rozmycia Gaussa ($\sigma = 1.0$) zlikwidowało całkowicie mikro-artefakty typu "szachownica" oraz pomogło w wygładzeniu stromych, schodkowych zboczy, sprawiając, że teren zaczął łagodnie i gładko wznosić się ku górze.

Dzięki takiemu podejściu uzyskaliśmy stabilną, trójwymiarową topografię, która w pełni reaguje na warstwę biologiczną (np. wysokie szczyty w biomach *mountains*, płaskie koryta w rzekach i stopniowe podnoszenie terenu od brzegu plaży). Wygenerowana macierz wysokości jest następnie przesyłana bezpośrednio jako wejście do Kroku 6 w celu wygenerowania warstw fizycznych w plikach `.mca`.



Od lewej: Wejściowa mapa powierzchni, Prawdziwa mapa wysokości, Wygenerowana i wygładzona mapa wysokości

Etap 5: Generowanie Roślinności (Vegetation)

Po wygenerowaniu mapy biomów oraz ukształtowania terenu, proces przechodzi do etapu umieszczania roślinności. Krok ten ma na celu ożywienie wygenerowanego świata, zachowując przy tym zgodność z charakterystyką każdego biomu.

Zbieranie statystyk

Pierwszym krokiem w procesie generowania roślinności jest zebranie odpowiednich statystyk. Algorytm analizuje oryginalne światy Minecraft z zestawu treningowego i tworzy histogramy występowania poszczególnych rodzajów flory w obrębie każdego biomu. Obliczone w ten sposób częstotliwości pozwalają ustalić dokładne prawdopodobieństwo pojawienia się konkretnej rośliny na danym obszarze.

Generowanie drzew

Istotnym rozwiązaniem wprowadzonym na tym etapie jest zastąpienie generowania pełnych struktur drzew ich sadzonkami (*saplings*). Zamiast uczyć algorytm budowy skomplikowanych i różnorodnych układów pni i liści, system w odpowiednich miejscach stawia właściwą sadzonkę, co znacząco zmniejsza złożoność obliczeniową i ułatwia tworzenie mapy.

Reguły walidacji roślin

Aby zapobiec nielogicznym generacjom i zachować spójność z mechanikami gry, algorytm stosuje zestaw twardych reguł sprawdzających otoczenie:

- Całkowicie zabronione jest generowanie jakichkolwiek roślin na blokach kamienia.
- W obrębie biomu pustynnego dozwolone jest generowanie wyłącznie kaktusów oraz martwych krzewów.
- Trzcina cukrowa może pojawić się tylko wtedy, gdy bezpośrednio sąsiaduje z blokiem wody znajdującym się dokładnie na tej samej wysokości, co blok podłoża trzciny.

Etap 6: Konwersja do formatu MCA (Eksport Świata)

Ostatnim etapem procesu jest konwersja wygenerowanych macierzy (wysokości, biomów oraz powierzchni) do natywnego formatu zapisu gry Minecraft – plików regionów (`.mca`). Pomimo tego, że nasz model generuje wyłącznie reprezentację 2.5D (czyli tylko pojedynczy widoczny blok powierzchniowy oraz ewentualną roślinność tuż nad nim), skrypt odpowiadający za ten krok sprawia, że świat staje się w pełni trójwymiarowy i grywalny.

Algorytm ten sztucznie dobudowuje brakujące warstwy podziemne (aż do najniższej współrzędnej $Y = 0$), opierając się na logicznych regułach zależnych od wygenerowanego biomu:

- **Podziemna podstawa:** Na samym dnie świata ($Y = 0$) umieszczana jest niezniszczalna skała macierzysta (*bedrock*), a cała przestrzeń od niej aż niemal do samej powierzchni wypełniana jest litym kamieniem (*stone*).
- **W pustyniach** tuż pod powierzchnią generowane są dodatkowe 3 bloki piasku (*sand*).
- **W rzekach**, jeśli na powierzchni wygenerowała się woda, pod nią umieszczana jest warstwa gliny (*clay*). Jeśli natomiast jest to brzeg rzeki – standardowo generowane są 3 bloki ziemi.
- **W oceanach** generowana jest prawdziwa głębia (kolumna wody sięgająca do 10 bloków w dół), a pod wodą układane jest piaszczyste dno.
- **W pozostałych biomach** (takich jak lasy czy równiny) pod powierzchnią standardowo umieszczane są 3 bloki ziemi (*dirt*).

Dzięki temu zabiegowi, nasza "płaska" mapa przekształca się w pełnoprawny, objętościowy świat 3D. Gracz może w nim normalnie kopać w dół, obserwując logiczne warstwy gleby, nie wpadając przy tym w nieskończoną pustkę gry.

Instrukcja: Jak wygenerować i uruchomić własny świat

Aby samodzielnie wygenerować obszar i zwiedzić go w oryginalnej grze Minecraft, należy postępować zgodnie z poniższymi instrukcjami:

1. Uruchom wirtualne środowisko, a następnie wywołaj główny skrypt generujący, podając wybraną nazwę dla świata. Proces predykcji wszystkich modeli zajmuje z reguły od 15 do 20 minut. W wierszu poleceń wpisz:

```
source venv/bin/activate
python generate_world.py [nazwa_świata]
```

2. Po zakończeniu działania skryptu, w głównym katalogu projektu w folderze `worlds` pojawi się nowy podfolder `[nazwa_świata]`. W jego wnętrzu znajdziesz katalog `regions` (zawierający gotowe pliki binarne gry) oraz `visualizations` (zawierający grafiki przedstawiające mapy z poszczególnych etapów generacji).
3. Otwórz grę Minecraft i stwórz nowy świat. Przejdź do dodatkowych opcji generowania (*More World Options*) i ustaw:
 - **World Type:** Superflat
 - Przejdź do: **Customise** → **Presets** → wybierz **The Void** → kliknij **Use Preset** → dwukrotnie zatwierdź przyciskiem **Done**.
 - **Game Mode:** Creative

Na koniec kliknij **Create New World**.

4. Gra początkowo zrespawnuje postać na wysokości zero ($Y = 0$). Ponieważ w wygenerowanych przez nas plikach cała przestrzeń od zera w górę wypełniona jest litym kamieniem, natychmiast odleć w górę o około 100 bloków (używając trybu Creative) lub użyj wbudowanej komendy np. `/tp ~ ~100 ~`.
5. Wyjdź z wygenerowanego świata (*Save and Quit to Title*). W systemie Windows wciśnij menu Start, wpisz w wyszukiwarce `%appdata%` i otwórz ten folder. Następnie przejdź do: `.minecraft` → `saves` → `[nazwa stworzonego przed chwilą świata]` → `region`.
6. Usuń wszystkie znajdujące się tam pliki (jeśli istnieją) i podmień je kopiując wszystkie pliki `.mca` z naszego wygenerowanego przed chwilą katalogu (`worlds/[nazwa_świata]/regions`).
7. Otwórz ponownie ten sam świat w grze Minecraft. Teren wygenerowany świat zostanie poprawnie załadowany i będzie gotowy do eksploracji.